SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECHNOLOGY (SVNIT), SURAT

Department of Artificial Intelligence

Intelligent Multi-Agent and Expert Systems (AI401)

Semester VII

EXPERIMENT 1

Knowledge Representation - Semantic Network with Inheritance

Student Name	Vedang Dubey
Roll No.	U23AI111
Lab No.	1
Date	5 August 2026

Aim

To represent knowledge as a semantic network of (subject, relation, object) triples and answer queries using property inheritance along is-a links, including exceptions.

Theory

A semantic network represents concepts as nodes and relationships as labelled, directed edges. The is-a relation organizes concepts into a taxonomy. A specific concept inherits properties from more general concepts unless a more specific exception overrides the inherited default.

For example, the network states that a bird can fly, but a penguin is a bird and has the exception cannot fly. Therefore, a query about a penguin must inspect the penguin node before consulting bird and animal. This is an example of default reasoning with exceptions and demonstrates the non-monotonic character of such reasoning: adding a more specific fact can invalidate a conclusion that was previously derived from a general default.

Algorithm

- Store the knowledge network as a list of (subject, relation, object) triples.
- For a node, repeatedly follow its is-a link until no parent exists to obtain the inheritance chain.
- For query(node, relation), traverse the chain from the most specific node to the most general node.
- At each level, check for a cannot exception before checking an ordinary value for the queried relation.
- Return the first applicable value; if the chain is exhausted, return unknown.

Procedure and Result

The following implementation uses Python. The knowledge base contains animal, fish, shark, bird, penguin, and their properties. The program prints the triples, the inheritance chain, the answers to representative queries, and the required exercise results.

Step 1 - Knowledge Base and Inheritance Chain

The triples used in the semantic network are:

Subject	Relation	Object
animal	has	skin
animal	can	breathe
fish	is-a	animal
fish	has	gills
shark	is-a	fish
bird	is-a	animal
bird	can	fly
bird	has	feathers
penguin	is-a	bird
penguin	cannot	fly
penguin	can	swim

```
python semantic_network.py

$ python semantic_network.py

=== KNOWLEDGE BASE (TRIPLES) ===
('animal', 'has', 'skin')
('animal', 'can', 'breathe')
('fish', 'is-a', 'animal')
('fish', 'has', 'gills')
('shark', 'is-a', 'fish')
('bird', 'is-a', 'animal')
('bird', 'can', 'fly')
('bird', 'has', 'feathers')
('penguin', 'is-a', 'bird')
('penguin', 'cannot', 'fly')
('penguin', 'can', 'swim')

=== INHERITANCE CHAIN ===
shark: shark -> fish -> animal
penguin: penguin -> bird -> animal
```

Snapshot 1 - Executed program showing the knowledge base and inheritance chains.

Step 2 - Query Processing

The query procedure walks from the most specific concept upward. Thus, for penguin, the order is penguin -> bird -> animal. The penguin exception cannot fly is checked before bird can fly, so the exception wins.

```
python semantic_network.py

=== QUERIES ===
query('shark', 'has') -> has gills
query('shark', 'can') -> can breathe
query('penguin', 'can') -> cannot fly
query('penguin', 'has') -> has feathers

Interpretation: penguin's local exception overrides bird's default.
```

Snapshot 2 - Query results, including the penguin exception.

Step 3 - Source Code

```
# AI401 - Experiment 1: Semantic Network with Inheritance

triples = [
    ("animal", "has", "skin"),
    ("animal", "can", "breathe"),
    ("fish", "is-a", "animal"),
    ("fish", "has", "gills"),
    ("shark", "is-a", "fish"),
    ("bird", "is-a", "animal"),
    ("bird", "can", "fly"),
    ("bird", "has", "feathers"),
    ("penguin", "is-a", "bird"),
    ("penguin", "cannot", "fly"),
    ("penguin", "can", "swim"),
]


def parents_of(node):
    return [obj for subj, rel, obj in triples if subj == node and rel == "is-a"]


def inheritance_chain(node):
    chain = [node]
    current = node
    visited = {node}
    while True:
        parents = parents_of(current)
```

```
        if not parents:
            break
        # For the base single-parent network, follow the first parent.
        current = parents[0]
        if current in visited:
            break
        chain.append(current)
        visited.add(current)
    return chain


def query(node, relation):
    """Most-specific-first lookup. Exceptions override inherited defaults."""
    for current in inheritance_chain(node):
        # Check exception at this level before normal values.
        for subj, rel, obj in triples:
            if subj == current and rel == "cannot" and relation == "can":
                return f"cannot {obj}"
        for subj, rel, obj in triples:
            if subj == current and rel == relation:
                return f"{relation} {obj}"
    return "unknown"


print("=== KNOWLEDGE BASE (TRIPLES) ===")
for triple in triples:
    print(triple)

print("\n=== INHERITANCE CHAIN ===")
for node in ["shark", "penguin"]:
    print(f"{node}: {' -> '.join(inheritance_chain(node))}")

print("\n=== QUERIES ===")
queries = [
    ("shark", "has"),
    ("shark", "can"),
    ("penguin", "can"),
    ("penguin", "has"),
    ("penguin", "can", "swim"),
]
# The first four use the required query(node, relation) API.
for node, relation, *rest in queries:
    if rest:
        print(f"query({node!r}, {relation!r}) -> {query(node, relation)}")
    else:
        print(f"query({node!r}, {relation!r}) -> {query(node, relation)}")


# Exercise 8: add ostrich.
print("\n=== EXERCISE 8: OSTRICH ===")
triples.extend([
    ("ostrich", "is-a", "bird"),
    ("ostrich", "cannot", "fly"),
    ("ostrich", "can", "run fast"),
])
print(f"ostrich chain: {' -> '.join(inheritance_chain('ostrich'))}")
print(f"query('ostrich', 'can') -> {query('ostrich', 'can')}")
print(f"local fact: ('ostrich', 'can', 'run fast') -> {('ostrich', 'can', 'run fast') in triples}")
print(f"query('ostrich', 'has') -> {query('ostrich', 'has')}")


# Exercise 9: frames.
print("\n=== EXERCISE 9: FRAME REPRESENTATION ===")
frames = {
    "animal": {"ako": None, "has": "skin", "can": "breathe"},
    "fish": {"ako": "animal", "has": "gills"},
```

```
    "shark": {"ako": "fish"},
    "bird": {"ako": "animal", "can": "fly", "has": "feathers"},
    "penguin": {"ako": "bird", "cannot": "fly", "can": "swim"},
    "ostrich": {"ako": "bird", "cannot": "fly", "can": "run fast"},
}


def frame_query(frame_name, relation):
    current = frame_name
    visited = set()
    while current is not None and current not in visited:
        visited.add(current)
        frame = frames[current]
        if frame.get("cannot") is not None and relation == "can":
            return f"cannot {frame["cannot"]}"
        if relation in frame:
            return f"{relation} {frame[relation]}"
        current = frame.get("ako")
    return "unknown"

print("Frames:")
for name, slots in frames.items():
    print(f"{name}: {slots}")
print(f"frame_query('penguin', 'can') -> {frame_query('penguin', 'can')}")
print(f"frame_query('shark', 'has') -> {frame_query('shark', 'has')}")
print(f"frame_query('ostrich', 'can') -> {frame_query('ostrich', 'can')}")
print(f"frame local slot ostrich.can -> {frames['ostrich']['can']}")


# Exercise 10: multiple inheritance / multiple is-a links.
print("\n=== EXERCISE 10: MULTIPLE PARENTS ===")
mi_triples = [
    ("animal", "can", "breathe"),
    ("bird", "is-a", "animal"),
    ("bird", "can", "fly"),
    ("water_animal", "can", "swim"),
    ("flying_animal", "can", "fly"),
    ("duck", "is-a", "bird"),
    ("duck", "is-a", "water_animal"),
    ("duck", "is-a", "flying_animal"),
]


def mi_parents(node):
    return [obj for subj, rel, obj in mi_triples if subj == node and rel == "is-a"]


def mi_chain_search(node, relation):
    # Breadth-first search gives the minimum inheritance distance first.
    # Parent order in the knowledge base breaks ties deterministically.
    queue = [(node, 0)]
    visited = {node}
    while queue:
        current, depth = queue.pop(0)
        for subj, rel, obj in mi_triples:
            if subj == current and rel == relation:
                return f"{relation} {obj}", current, depth
        for parent in mi_parents(current):
            if parent not in visited:
                visited.add(parent)
                queue.append((parent, depth + 1))
    return "unknown", None, None

print("Multiple-parent triples:")
for triple in mi_triples:
    print(triple)
```

```
print("Search policy: breadth-first; nearest ancestor first; knowledge-base order breaks ties.")
print(f"query('duck', 'can') -> {mi_chain_search('duck', 'can')}")
print(f"query('duck', 'has') -> {mi_chain_search('duck', 'has')}")
```

Observations

- The query query('penguin', 'can') returns cannot fly because the most specific node, penguin, contains an explicit exception that overrides bird's inherited default can fly.
- Exceptions must be checked before inherited values at every level; otherwise the algorithm would incorrectly stop at bird and conclude that a penguin can fly.
- The result demonstrates non-monotonic default reasoning. A general fact can support a conclusion, but adding a more specific exception can retract that conclusion for a particular instance.
- The shark example shows ordinary inheritance: shark inherits gills from fish and breathe from animal without storing these properties directly on shark.
- The unknown result in the multi-parent exercise shows that inheritance alone does not create properties that are absent from all reachable ancestors.

Exercise 8 - Extend the Network with an Ostrich

The network is extended with the triples (ostrich, is-a, bird), (ostrich, cannot, fly), and (ostrich, can, run fast). The query for can correctly reports the local exception cannot fly, while the local data structure also confirms that run fast is a direct ostrich capability. Feathers are inherited from bird.

```
python semantic_network.py

=== EXERCISE 8: OSTRICH ===
ostrich chain: ostrich -> bird -> animal
query('ostrich', 'can') -> cannot fly
local fact: ('ostrich', 'can', 'run fast') -> True
query('ostrich', 'has') -> has feathers

=== EXERCISE 9: FRAME REPRESENTATION ===
frame_query('penguin', 'can') -> cannot fly
frame_query('shark', 'has') -> has gills
frame local slot ostrich.can -> run fast
```

Snapshot 3 - Ostrich extension and frame-representation results.

Exercise 9 - Re-implement as Frames

The same knowledge is represented as a Python dictionary per concept. The ako slot acts as the frame equivalent of is-a. Other slots are property-value pairs. The inheritance lookup follows the ako pointer from the most specific frame upward and checks the local exception before inherited values.

```
frames = {
    "animal": {"ako": None, "has": "skin", "can": "breathe"},
    "fish": {"ako": "animal", "has": "gills"},
    "shark": {"ako": "fish"},
    "bird": {"ako": "animal", "can": "fly", "has": "feathers"},
    "penguin": {"ako": "bird", "cannot": "fly", "can": "swim"},
    "ostrich": {"ako": "bird", "cannot": "fly", "can": "run fast"},
}

def frame_query(frame_name, relation):
    current = frame_name
    visited = set()
    while current is not None and current not in visited:
        visited.add(current)
        frame = frames[current]
```

```
        if frame.get("cannot") is not None and relation == "can":
            return f"cannot {frame["cannot"]}"
        if relation in frame and relation != "ako":
            return f"{relation} {frame[relation]}"
        current = frame.get("ako")
    return "unknown"
```

The frame implementation produces the same inheritance behaviour as the triple-based network. For example, frame_query('penguin', 'can') returns cannot fly and frame_query('shark', 'has') returns has gills.

Exercise 10 - Multiple Parents

For multiple is-a links, a deterministic search policy is required. This implementation uses breadth-first search (BFS): the most specific node is checked first, then all parents at distance 1, then distance 2, and so on. If two parents are at the same distance, their order in the knowledge base is used as the tie-breaker.

Justification: BFS is preferable to blindly following the first parent because it finds the nearest applicable ancestor and avoids arbitrarily choosing a deeper property when a nearer parent has a matching property. Using the knowledge-base insertion order for ties keeps the result deterministic and reproducible.

```
python semantic_network.py

=== EXERCISE 10: MULTIPLE PARENTS ===
Search policy: breadth-first; nearest ancestor first; KB order breaks ties.

query('duck', 'can') -> ('can fly', 'bird', 1)
query('duck', 'has') -> ('unknown', None, None)

Justification: BFS avoids arbitrary deep-parent preference and finds the
nearest applicable ancestor; insertion order makes equal-depth results deterministic.
```

Snapshot 4 - Multiple-parent search and selected ordering policy.

Key Results

Query	Expected / Observed Result	Reason
query('shark', 'has')	has gills	Inherited from fish
query('shark', 'can')	can breathe	Inherited from animal
query('penguin', 'can')	cannot fly	Local exception overrides bird default
query('penguin', 'has')	has feathers	Inherited from bird
ostrich local fact	can run fast	Stored directly on ostrich
frame_query('penguin', 'can')	cannot fly	Same exception rule in frames
duck multiple-parent can query	can fly via bird	Nearest matching parent under BFS

Conclusion

The experiment demonstrates declarative knowledge representation using a semantic network and shows how inheritance can be combined with exceptions. The most-specific-first strategy makes local knowledge override general defaults, while the frame implementation shows that the same reasoning pattern can be expressed through slots and ako links. The multiple-parent extension further shows that inheritance semantics require an explicit and deterministic conflict-resolution policy.

Appendix - Complete Program

The complete executable Python program used to generate the snapshots and results is included below. It can be copied into a file named semantic_network.py and run with Python 3.

```
# AI401 - Experiment 1: Semantic Network with Inheritance

triples = [
    ("animal", "has", "skin"),
    ("animal", "can", "breathe"),
    ("fish", "is-a", "animal"),
    ("fish", "has", "gills"),
    ("shark", "is-a", "fish"),
    ("bird", "is-a", "animal"),
    ("bird", "can", "fly"),
    ("bird", "has", "feathers"),
    ("penguin", "is-a", "bird"),
    ("penguin", "cannot", "fly"),
    ("penguin", "can", "swim"),
]


def parents_of(node):
    return [obj for subj, rel, obj in triples if subj == node and rel == "is-a"]


def inheritance_chain(node):
    chain = [node]
    current = node
    visited = {node}
    while True:
        parents = parents_of(current)
        if not parents:
            break
        # For the base single-parent network, follow the first parent.
        current = parents[0]
        if current in visited:
            break
        chain.append(current)
        visited.add(current)
    return chain


def query(node, relation):
    """Most-specific-first lookup. Exceptions override inherited defaults."""
    for current in inheritance_chain(node):
        # Check exception at this level before normal values.
        for subj, rel, obj in triples:
            if subj == current and rel == "cannot" and relation == "can":
                return f"cannot {obj}"
        for subj, rel, obj in triples:
            if subj == current and rel == relation:
                return f"{relation} {obj}"
    return "unknown"


print("=== KNOWLEDGE BASE (TRIPLES) ===")
for triple in triples:
    print(triple)

print("\n=== INHERITANCE CHAIN ===")
for node in ["shark", "penguin"]:
    print(f"{node}: {' -> '.join(inheritance_chain(node))}")

print("\n=== QUERIES ===")
queries = [
    ("shark", "has"),
    ("shark", "can"),
```

```
    ("penguin", "can"),
    ("penguin", "has"),
    ("penguin", "can", "swim"),
]
# The first four use the required query(node, relation) API.
for node, relation, *rest in queries:
    if rest:
        print(f"query({node!r}, {relation!r}) -> {query(node, relation)}")
    else:
        print(f"query({node!r}, {relation!r}) -> {query(node, relation)}")


# Exercise 8: add ostrich.
print("\n=== EXERCISE 8: OSTRICH ===")
triples.extend([
    ("ostrich", "is-a", "bird"),
    ("ostrich", "cannot", "fly"),
    ("ostrich", "can", "run fast"),
])
print(f"ostrich chain: {' -> '.join(inheritance_chain('ostrich'))}")
print(f"query('ostrich', 'can') -> {query('ostrich', 'can')}")
print(f"local fact: ('ostrich', 'can', 'run fast') -> {('ostrich', 'can', 'run fast') in triples}")
print(f"query('ostrich', 'has') -> {query('ostrich', 'has')}")


# Exercise 9: frames.
print("\n=== EXERCISE 9: FRAME REPRESENTATION ===")
frames = {
    "animal": {"ako": None, "has": "skin", "can": "breathe"},
    "fish": {"ako": "animal", "has": "gills"},
    "shark": {"ako": "fish"},
    "bird": {"ako": "animal", "can": "fly", "has": "feathers"},
    "penguin": {"ako": "bird", "cannot": "fly", "can": "swim"},
    "ostrich": {"ako": "bird", "cannot": "fly", "can": "run fast"},
}


def frame_query(frame_name, relation):
    current = frame_name
    visited = set()
    while current is not None and current not in visited:
        visited.add(current)
        frame = frames[current]
        if frame.get("cannot") is not None and relation == "can":
            return f"cannot {frame["cannot"]}"
        if relation in frame:
            return f"{relation} {frame[relation]}"
        current = frame.get("ako")
    return "unknown"

print("Frames:")
for name, slots in frames.items():
    print(f"{name}: {slots}")
print(f"frame_query('penguin', 'can') -> {frame_query('penguin', 'can')}")
print(f"frame_query('shark', 'has') -> {frame_query('shark', 'has')}")
print(f"frame_query('ostrich', 'can') -> {frame_query('ostrich', 'can')}")
print(f"frame local slot ostrich.can -> {frames['ostrich']['can']}")


# Exercise 10: multiple inheritance / multiple is-a links.
print("\n=== EXERCISE 10: MULTIPLE PARENTS ===")
mi_triples = [
    ("animal", "can", "breathe"),
    ("bird", "is-a", "animal"),
    ("bird", "can", "fly"),
    ("water_animal", "can", "swim"),
```

```
    ("flying_animal", "can", "fly"),
    ("duck", "is-a", "bird"),
    ("duck", "is-a", "water_animal"),
    ("duck", "is-a", "flying_animal"),
]


def mi_parents(node):
    return [obj for subj, rel, obj in mi_triples if subj == node and rel == "is-a"]


def mi_chain_search(node, relation):
    # Breadth-first search gives the minimum inheritance distance first.
    # Parent order in the knowledge base breaks ties deterministically.
    queue = [(node, 0)]
    visited = {node}
    while queue:
        current, depth = queue.pop(0)
        for subj, rel, obj in mi_triples:
            if subj == current and rel == relation:
                return f"{relation} {obj}", current, depth
        for parent in mi_parents(current):
            if parent not in visited:
                visited.add(parent)
                queue.append((parent, depth + 1))
    return "unknown", None, None

print("Multiple-parent triples:")
for triple in mi_triples:
    print(triple)
print("Search policy: breadth-first; nearest ancestor first; knowledge-base order breaks ties.")
print(f"query('duck', 'can') -> {mi_chain_search('duck', 'can')}")
print(f"query('duck', 'has') -> {mi_chain_search('duck', 'has')}")
```

Matplotlib Sketches and Graphical Analysis

The following plots were generated using Python and Matplotlib from the same semantic-network knowledge base and exercise results. They provide visual representations of the taxonomy, inheritance depth, query resolution, exception precedence, frame representation, multiple-parent search order, and the final query matrix.

Figure 1: Semantic Network Topology with Inheritance and Exceptions

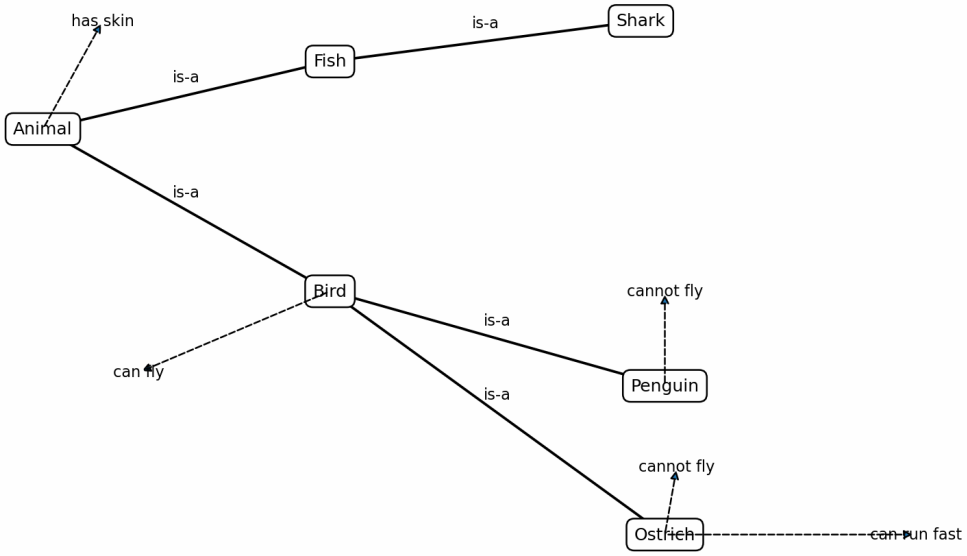


Figure 1: Semantic network topology with is-a links and property/exception edges. The arrows show how specific concepts inherit from more general concepts.

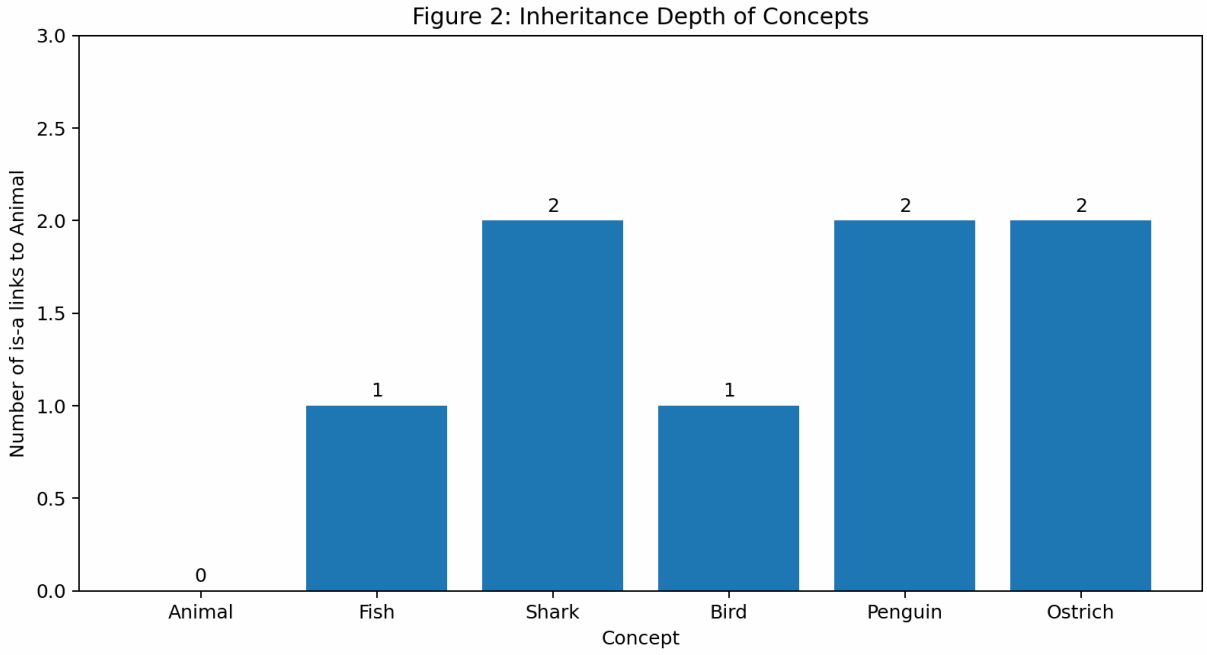


Figure 2: Inheritance depth of each concept. A larger value means more is-a links must be traversed before reaching Animal.

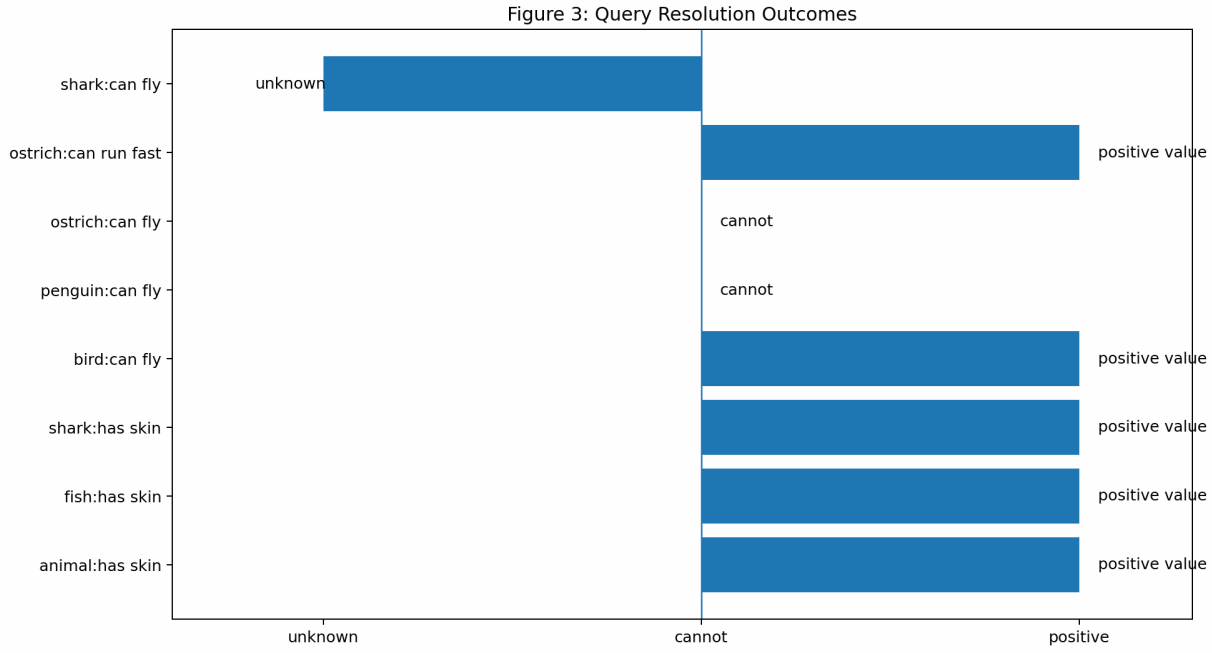


Figure 3: Query resolution outcomes. Positive results correspond to inherited or local properties; cannot indicates an exception; unknown means no value exists in the complete chain.

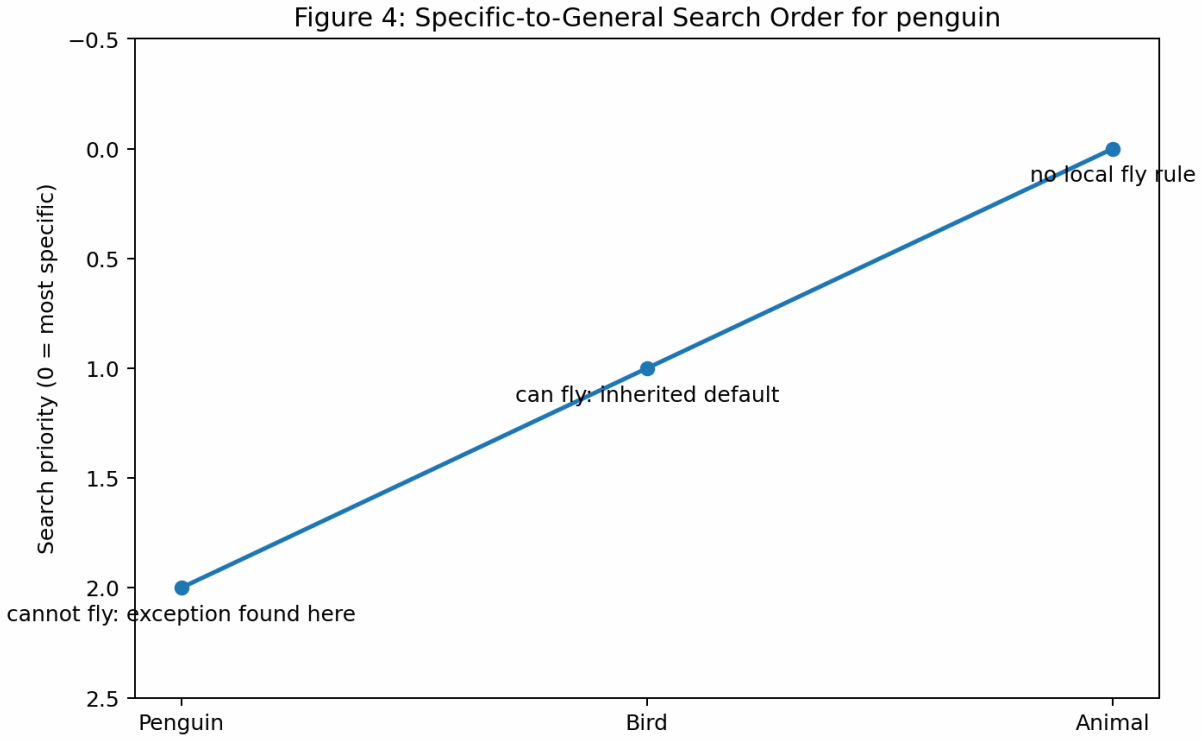


Figure 4: Specific-to-general search order for penguin. The local cannot fly exception is encountered before the inherited bird can fly default, so the exception wins.

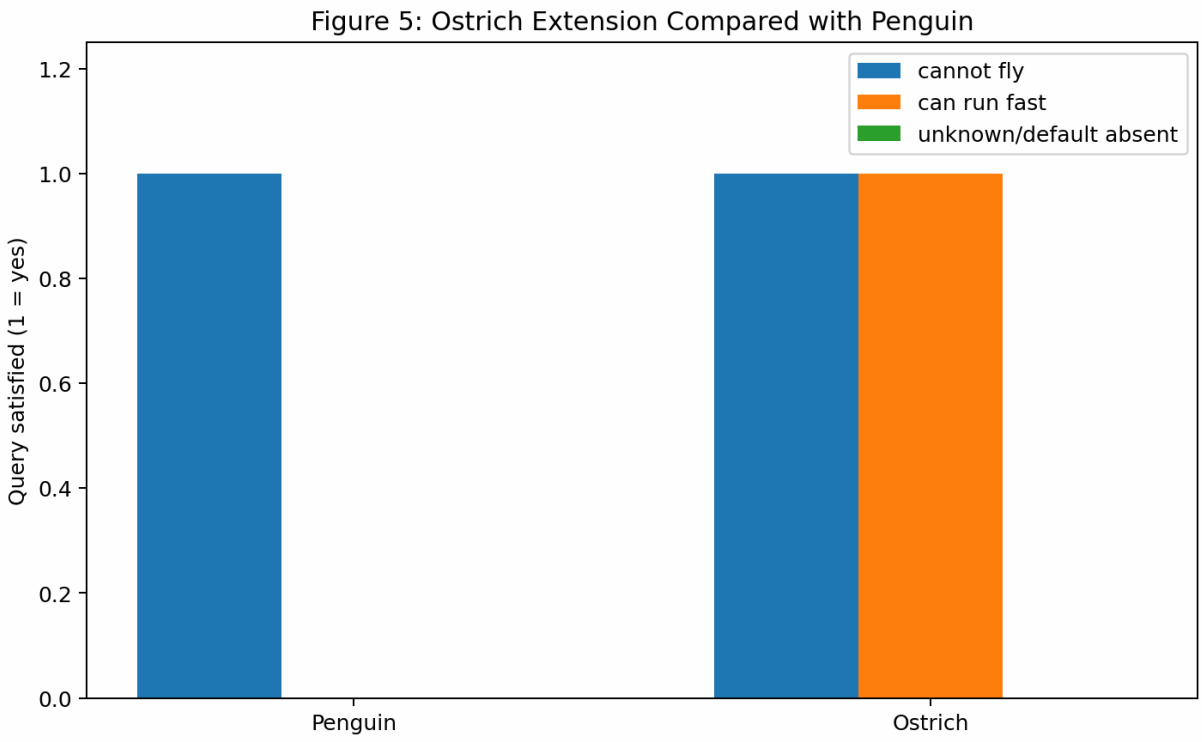


Figure 5: Exercise 8 comparison. Ostrich inherits bird properties but locally overrides flight and additionally has the run-fast property.

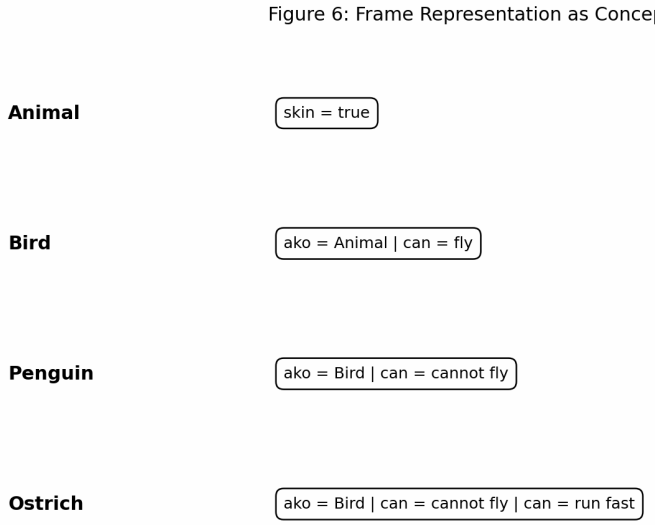


Figure 6: Exercise 9 frame representation. Each concept is a frame with slots/fillers and an ako link to its parent frame.

Figure 7: Multiple-Parent Inheritance and Search Order

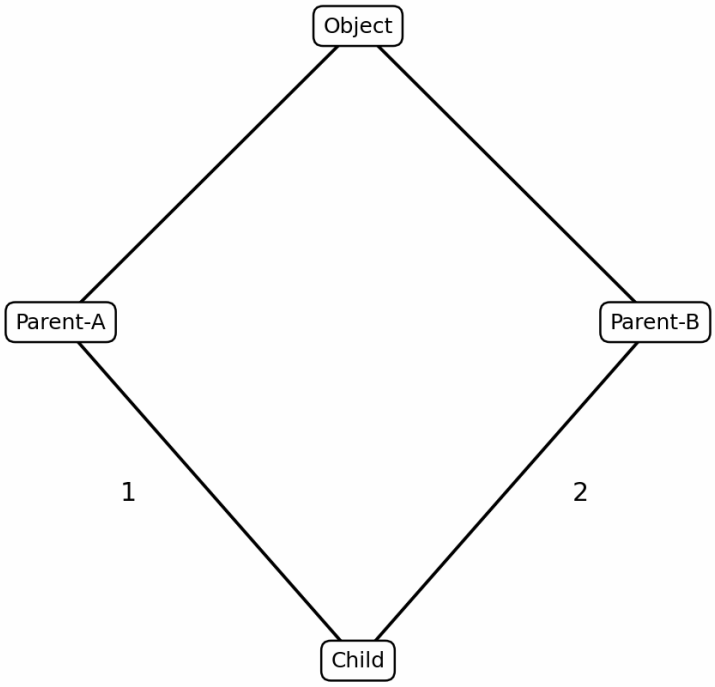


Figure 7: Exercise 10 multiple-parent inheritance. A deterministic left-to-right parent order is used so that conflicts have a reproducible resolution policy.

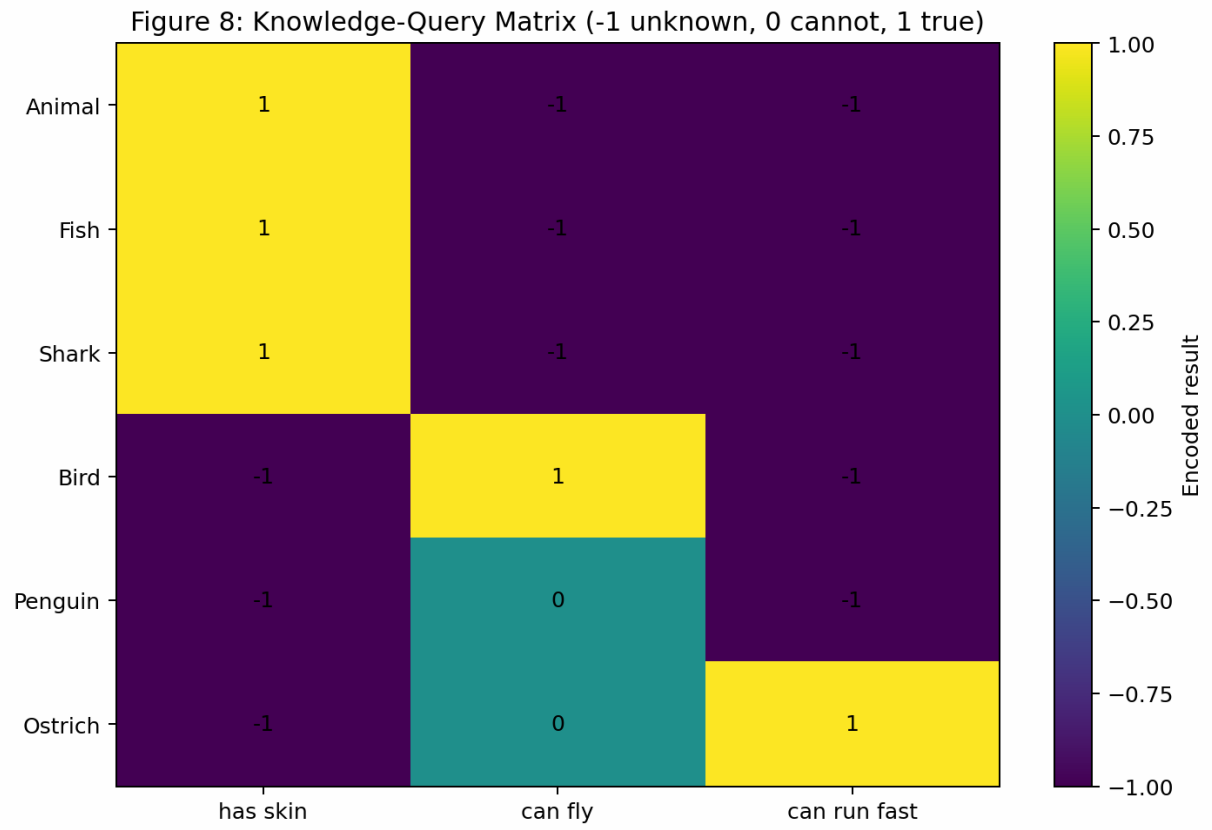


Figure 8: Complete query matrix for representative concepts. Values are encoded as 1 = true/positive, 0 = cannot/exception, and -1 = unknown.

Graphical Observations

- The semantic-network sketch makes the inheritance hierarchy explicit: Shark reaches Animal through Fish, while Penguin and Ostrich reach Animal through Bird.
- The inheritance-depth plot shows why lookup is naturally implemented as a chain traversal: more specific concepts require more ancestor checks.
- The query-outcome and matrix plots clearly separate positive knowledge, explicit exceptions, and unknown knowledge. Unknown is not treated as false.
- The exception-precedence plot illustrates non-monotonic default reasoning: adding a more specific exception can change the answer to a query without deleting the inherited bird rule.
- The frame plot shows that the same declarative knowledge can be represented with concept-specific slots and an ako relation instead of explicit triples.
- The multiple-parent sketch motivates a deterministic conflict policy. A left-to-right parent order is simple, reproducible, and easy to implement; in a larger expert system, a domain-specific priority or linearization such as C3-style ordering could also be used.